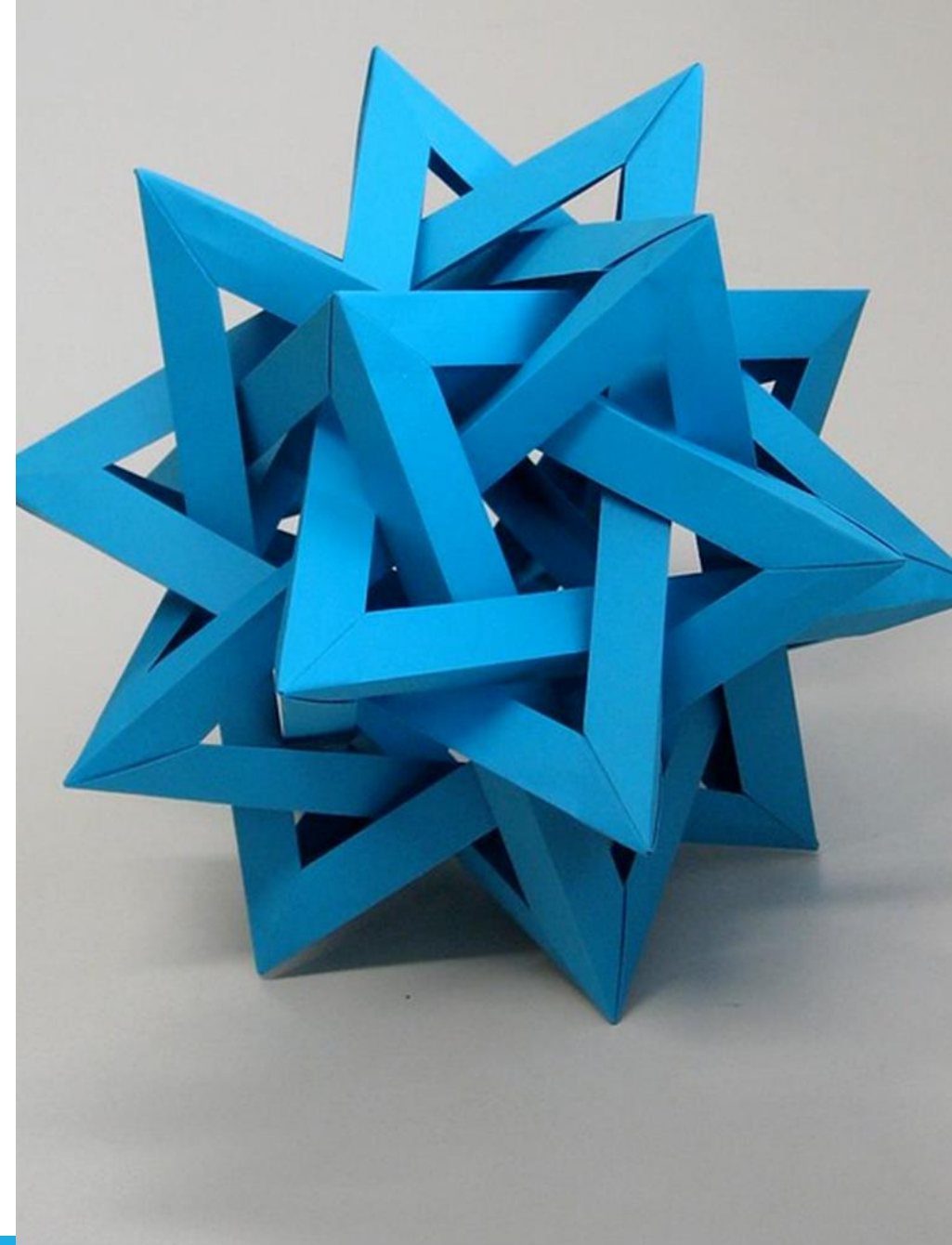


LAB 8

LISTS AND TABLES



Recommended Exercises

- **8.1.2**
- **8.1.4**
- **8.2.1**
- **8.2.2**

Exercise 8.1.2

- Read request from Lab text..

Exercise 8.1.2 (cont'd)

- Simpler than it looks
- “*Let's play*” with slicing
 - The function must create and return a new list who has:
 - **As first element the last one of the original list (how do you identify it?)**
 - **Followed by the remaining elements of the original list, EXCLUDING the last one (indexes from ... to)**
- Of course , it can also be done without slicing (think about it, in that case you might not need to create a new list – in-place solution)

Exercise 8.1.4

- Read request from Lab text..

Exercise 8.1.4 (cont'd)

- “Warm-up” exercise on tables
 - Use as much as possible what we’ve seen and written in class
 - **Initialize the table with zeros (0);**
 - N rows of 0s, appended to the table
 - **fill the table with ones (1);**
 - Try solving this with two nested loops, assigning each individual element (although it can also be done in the same way as the previous point)
 - **fill the table alternating 0s and 1s in a checkerboard pattern;**
 - Which “formula” and/or relationship characterizes the indexes of the cells to be filled with a 0 (or a 1)?

Exercise 8.1.4 (cont'd)

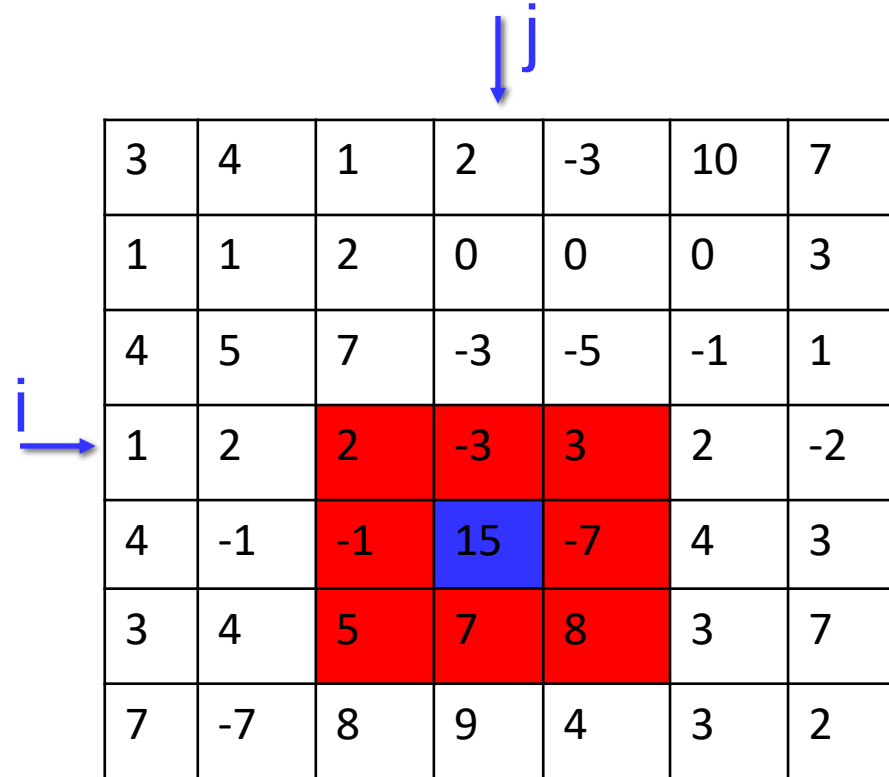
- Fill top+bottom rows with 0s, leaving the rest of the table unchanged;
 - How do you identify these lines (indexes...)?
- Fill leftmost+rightmost columns with 1s, leaving the rest of the table unchanged;
 - How do you identify these columns (indexes...)?
- calculate and print the sum of all the elements.
 - Seen in class...
- After each operation, print the table
 - Use the **printTab ()** function seen in class

Exercise 8.2.1

- Read request from Lab text..

Exercise 8.2.1 (cont'd)

- General case (e.g. $i = 4, j = 3$):

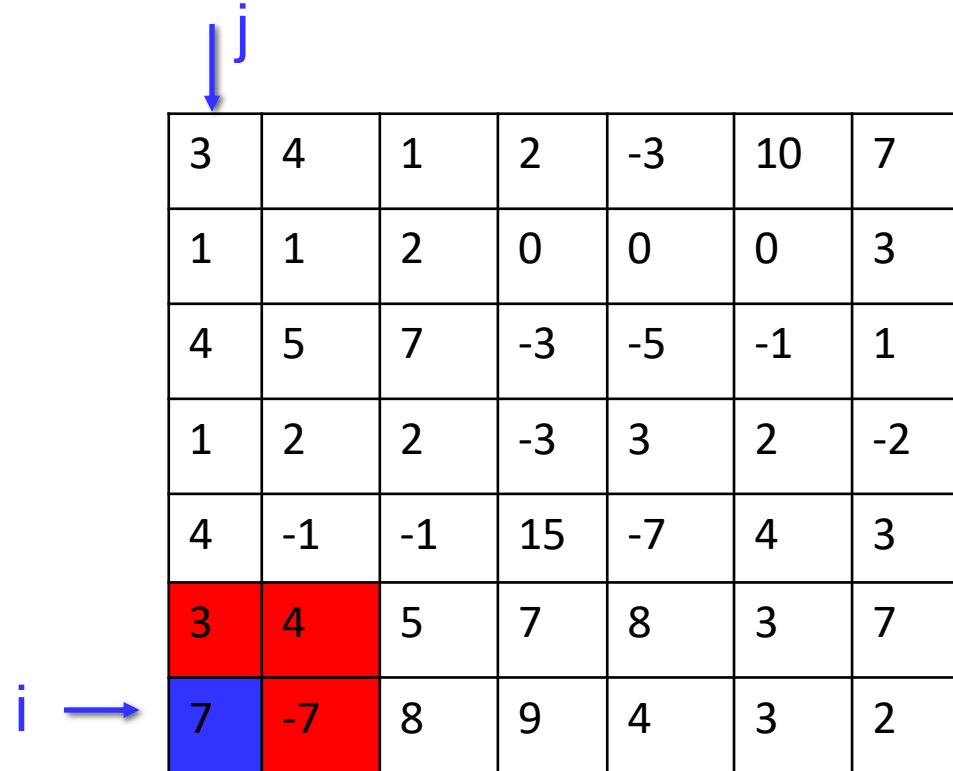


3	4	1	2	-3	10	7
1	1	2	0	0	0	3
4	5	7	-3	-5	-1	1
1	2	2	-3	3	2	-2
4	-1	-1	15	-7	4	3
3	4	5	7	8	3	7
7	-7	8	9	4	3	2

- Average: $[2 + (-3) + 3 + (-1) + (-7) + 5 + 7 + 8]/8 = 1.75$

Exercise 8.2.1 (cont'd)

- Special case (e.g. $i = 6, j = 0$):



3	4	1	2	-3	10	7
1	1	2	0	0	0	3
4	5	7	-3	-5	-1	1
1	2	2	-3	3	2	-2
4	-1	-1	15	-7	4	3
3	4	5	7	8	3	7
7	-7	8	9	4	3	2

- Average: $[3 + 4 + (-7)] / 3 = 0$
 - Note: there are other special cases with number of neighbors $\neq 3$

Exercise 8.2.1 (cont'd)

- Hint : it is enough to check that the indices do not exceed the valid ranges
 - Row index from 0 to `len(values)`
 - Column index from 0 to `len(values[i])` (or `len(values[0])` assuming all rows are equally long)
- write a `main()` that calculates the average for three different positions in a 4x4 table : (0,0), (1,2) and (3,2):

```
def main():  
    table = [] # fill it as you wish...  
    print("The average at (0,0) is", neighborAverage (table, 0, 0))  
    print("The average at (1,2) is", neighborAverage (table, 1, 2))  
    print("The average at (3,2) is", neighborAverage (table, 3, 2))
```

Exercise 8.2.2

- Read request from Lab text..

Exercise 8.2.2 (cont'd)

- For reading we can use the pattern seen in class :

For each row (1...4)

iterate 4 times

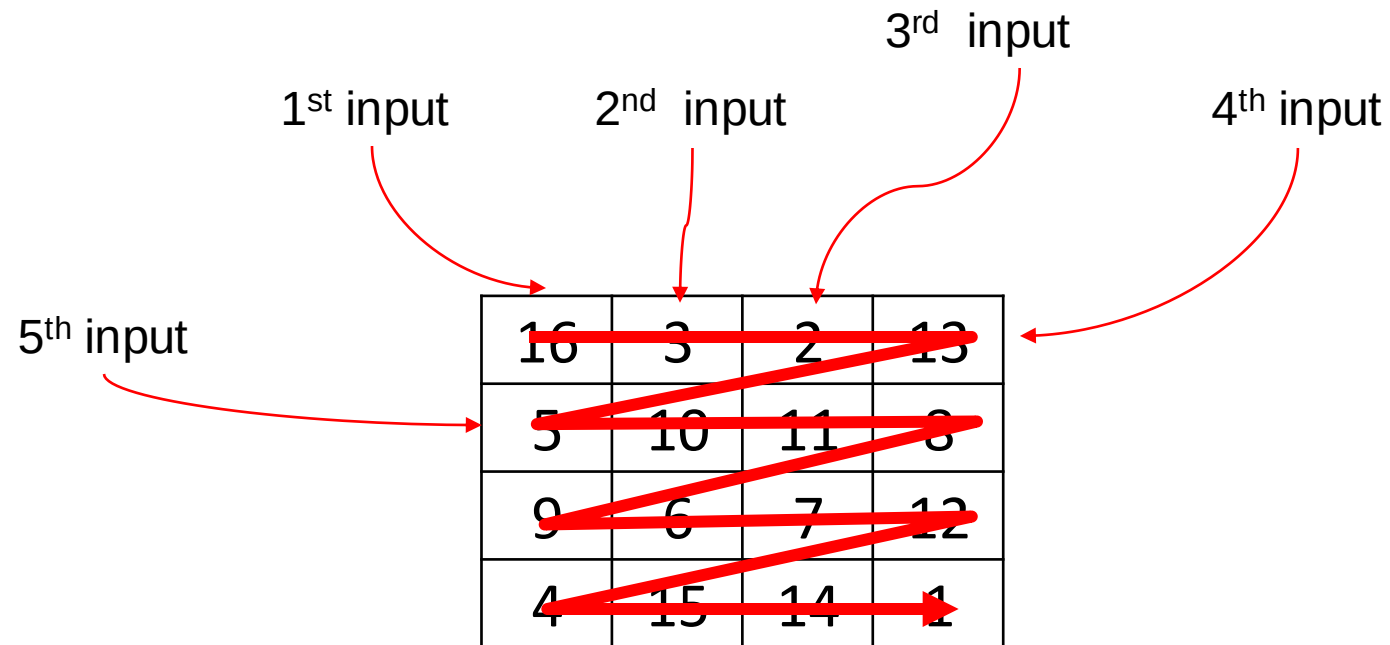
append() to the row a value read as input

append() the row to the table

- Scheme:
 - 1) read a complete row
 - 2) add the row to the table
- **TIP:** For testing purposes, you can also use a table initialized “by hand ” in the code.

Exercise 8.2.2 (cont'd)

- Reading the square , whose size is fixed and equal to 4x4:



Exercise 8.2.2 (cont'd)

- Check #1: Knowing the size of the square we know **which** values must be present .
- Therefore, we have to check if all numbers from 1 to 16 are present in one of the rows from the table .
- Is it necessary to check that they appear only once? Why?

Exercise 8.2.2 (cont'd)

- Check #2: If the sum of all rows, columns and diagonals must to be same , we can take any one (e.g. the **first row**) as a reference.
- After that, we have to check that *all the others rows , all columns and both diagonals have the same sum*. Only if this condition is **always verified** the square will be magic.
- *In other words, even if just one row /column/ diagonal has different sum , the square **is not** magic .*

Exercise 8.2.2 (cont'd)

- So we are in a case of the classic pattern in which a condition must always be verified, already seen in class

- 1) Calculate a total reference **sum** (easier using one row)
- 2) Initialize a flag **magic=True**
- 3) If any of the sums (rows / columns / diagonals) is different from **sum**, then **magic =False**

Exercise 8.2.2 (cont'd)

- **NOTE** : Even though the square only contains 4 elements in each row/column/diagonal (and therefore we could sum the values explicitly) , it's much better to use a loop to calculate them (this makes your code reusable also for magic squares of different size):
 - **Sum of every column** : two nested loops
 - **Sum of every row** : can be done with a single loop (one row is a list!)
 - **Sum of diagonals** : which indices combination identify the elements of the “diagonal ”? How many there are? Single loop or two nested loops?